

PRINCIPAL COMPONENT REGRESSION WITHOUT PRINCIPAL COMPONENT ANALYSIS

Roy Frostig ¹, Cameron Musco ², Christopher Musco ², Aaron Sidford ³

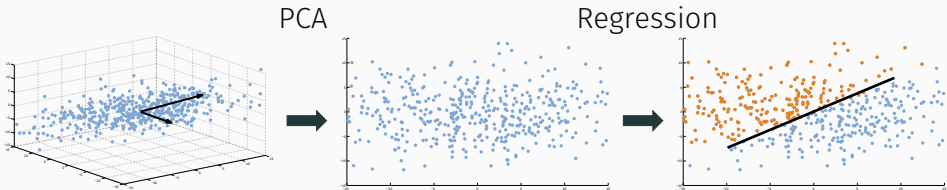
¹Stanford, ²MIT, ³Microsoft Research

Paper, slides, and template code available at
chrismusco.com

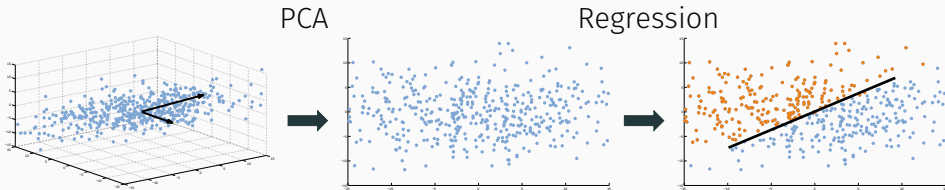
Simple, robust algorithms for principal component regression.

Principal Component Regression (PCR) =
Principal Component Analysis (unsupervised)
+
Linear Regression (supervised)

OUR APPROACH: SKIP THE DIMENSIONALITY REDUCTION



OUR APPROACH: SKIP THE DIMENSIONALITY REDUCTION

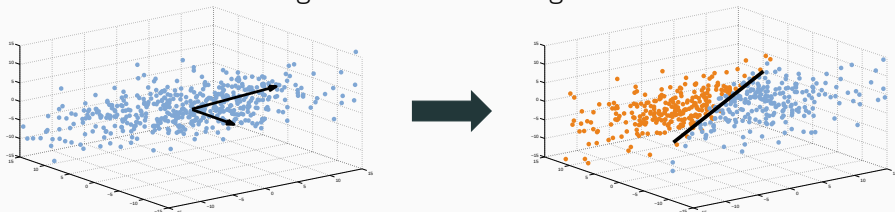


Regression is cheap (fast iterative or stochastic methods).

PCA is a major computational bottleneck.

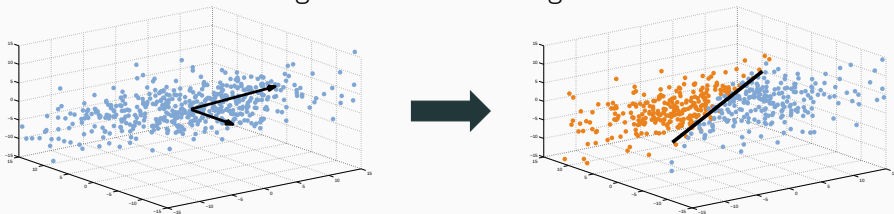
OUR APPROACH: SKIP THE DIMENSIONALITY REDUCTION

Single-shot iterative algorithm



OUR APPROACH: SKIP THE DIMENSIONALITY REDUCTION

Single-shot iterative algorithm



Final algorithm just uses a few applications of any fast, black-box regression routine.

Standard Regression:

Given: $\mathbf{A}$, $\mathbf{b}$

Solve: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|^2$

Standard Regression:

Given: $\mathbf{A}$, $\mathbf{b}$

Solve: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|^2$

Principal Component Regression:

Given: $\mathbf{A}$, $\mathbf{b}$, λ

Solve: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{A}_{\lambda}\mathbf{x} - \mathbf{b}\|^2$

Standard Regression:

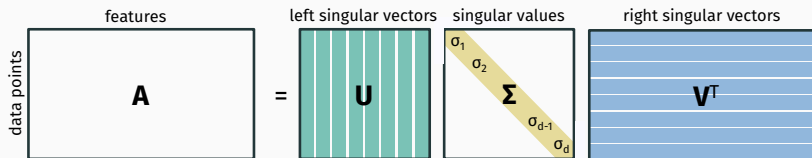
Given: $\mathbf{A}$, $\mathbf{b}$

Solve: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|^2$

Principal Component Regression:

Given: $\mathbf{A}$, $\mathbf{b}$, λ

Solve: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{A}_{\lambda} \mathbf{x} - \mathbf{b}\|^2$



Standard Regression:

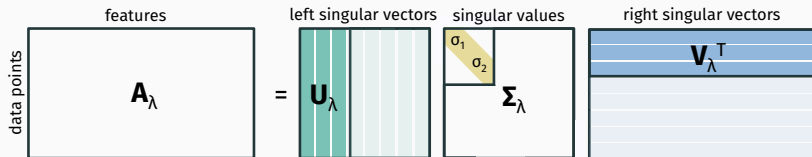
Given: $\mathbf{A}$, $\mathbf{b}$

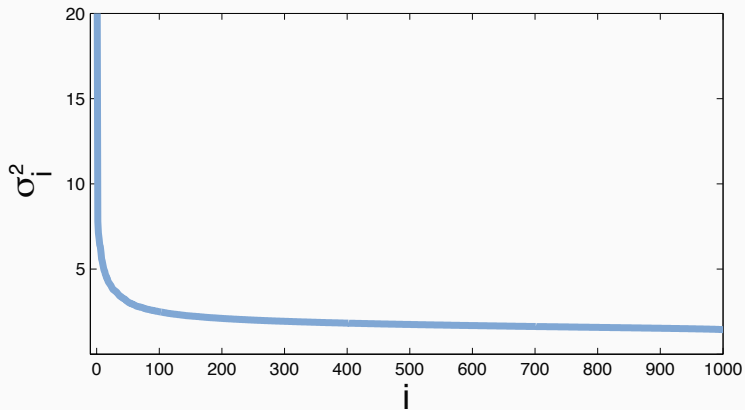
Solve: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|^2$

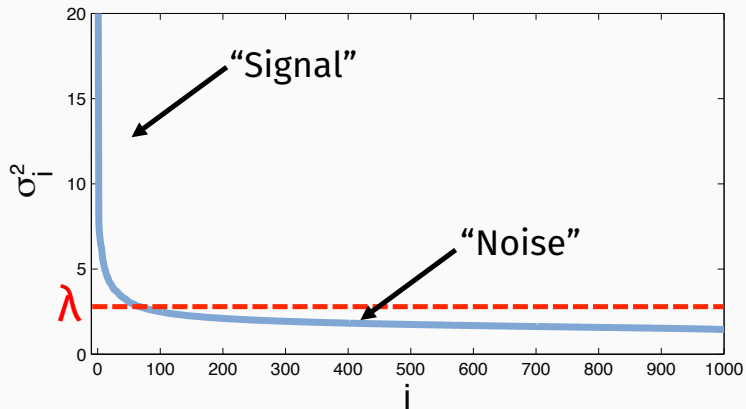
Principal Component Regression:

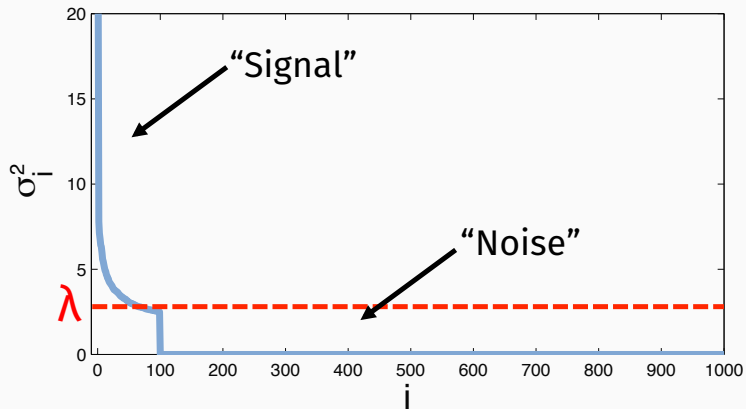
Given: $\mathbf{A}$, $\mathbf{b}$, λ

Solve: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{A}_{\lambda}\mathbf{x} - \mathbf{b}\|^2$









Principal Component Regression:

$$\text{Goal: } \mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{A}_{\lambda} \mathbf{x} - \mathbf{b}\|^2$$

$$\text{Solution: } \mathbf{x} = (\mathbf{A}_{\lambda}^T \mathbf{A}_{\lambda})^{-1} \mathbf{A}_{\lambda}^T \mathbf{b}$$

Cost of computing A_λ (PCA):

$$\approx O(\text{nnz}(\mathbf{A})k + dk^2).$$

Cost of computing A_λ (PCA):

$$\approx O(\text{nnz}(\mathbf{A})k + dk^2).$$

k is the number of principal components with value $> \lambda$.

Cost of computing A_λ (PCA):

$$\approx O(\text{nnz}(\mathbf{A})k + dk^2).$$

k is the number of principal components with value $> \lambda$.

Cost of evaluating $\mathbf{x} = (\mathbf{A}_\lambda^T \mathbf{A}_\lambda)^{-1} \mathbf{A}_\lambda^T \mathbf{b}$ (regression):

$$\approx O(\text{nnz}(\mathbf{A}) \cdot \sqrt{\kappa}).$$

Cost of computing $\mathbf{A}_\lambda$ (PCA):

$$\approx O(\text{nnz}(\mathbf{A})k + dk^2).$$

k is the number of principal components with value $> \lambda$.

Cost of evaluating $\mathbf{x} = (\mathbf{A}_\lambda^T \mathbf{A}_\lambda)^{-1} \mathbf{A}_\lambda^T \mathbf{b}$ (regression):

$$\approx O(\text{nnz}(\mathbf{A}) \cdot \sqrt{\kappa}).$$

No dependence on k . Even better runtimes achievable via stochastic methods.

Goal: Remove bottleneck dependence on k

Optimistic observation: PCA computes **too much information**.

Optimistic observation: PCA computes **too much information**.

$$(\mathbf{A}_\lambda^T \mathbf{A}_\lambda)^{-1} \mathbf{A}_\lambda^T \mathbf{b} =$$

Optimistic observation: PCA computes too much information.

$$(\mathbf{A}_\lambda^T \mathbf{A}_\lambda)^{-1} \mathbf{A}_\lambda^T \mathbf{b} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}_\lambda^T \mathbf{b}$$

Optimistic observation: PCA computes **too much information**.

$$(\mathbf{A}_\lambda^T \mathbf{A}_\lambda)^{-1} \mathbf{A}_\lambda^T \mathbf{b} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}_\lambda^T \mathbf{b}$$

Don't need to compute $\mathbf{A}_\lambda$ (which incurs a k dependence) as long as we can apply it to a *single vector* efficiently.

It's very often more efficient to *apply* a matrix function once than compute the matrix function explicitly.

It's very often more efficient to *apply* a matrix function once than compute the matrix function explicitly.

- $(A^T A)x$, $(A^T A)^2 x$, or $(A^T A)^3 x$

It's very often more efficient to *apply* a matrix function once than compute the matrix function explicitly.

- $(A^T A)x$, $(A^T A)^2 x$, or $(A^T A)^3 x$
- $A^{-1}x$

It's very often more efficient to *apply* a matrix function once than compute the matrix function explicitly.

- $(\mathbf{A}^T \mathbf{A})\mathbf{x}$, $(\mathbf{A}^T \mathbf{A})^2\mathbf{x}$, or $(\mathbf{A}^T \mathbf{A})^3\mathbf{x}$
- $\mathbf{A}^{-1}\mathbf{x}$
- $\exp(\mathbf{A}) \dots$ many more

It's very often more efficient to *apply* a matrix function once than compute the matrix function explicitly.

- $(\mathbf{A}^T \mathbf{A})\mathbf{x}$, $(\mathbf{A}^T \mathbf{A})^2\mathbf{x}$, or $(\mathbf{A}^T \mathbf{A})^3\mathbf{x}$
- $\mathbf{A}^{-1}\mathbf{x}$
- $\exp(\mathbf{A}) \dots$ many more

Why not $\mathbf{A}_\lambda^T$?

Theorem (Main Result)

There's an algorithm that approximately applies $\mathbf{A}_\lambda^T$ to any vector $\mathbf{b}$ using $\approx \log(1/\epsilon)$ well conditioned linear system solutions.

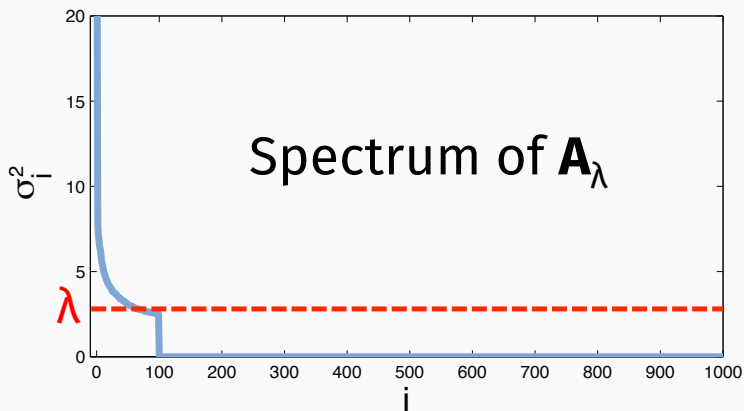
Theorem (Main Result)

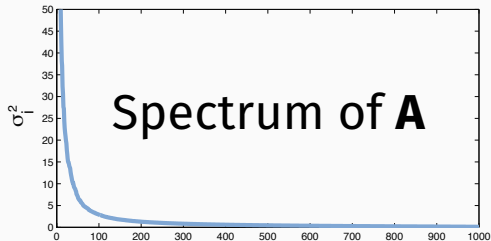
There's an algorithm that approximately applies $\mathbf{A}_\lambda^T$ to any vector $\mathbf{b}$ using $\approx \log(1/\epsilon)$ well conditioned linear system solutions.

PCR in $\approx O(\text{nnz}(\mathbf{A}) \cdot \sqrt{\kappa})$ time.

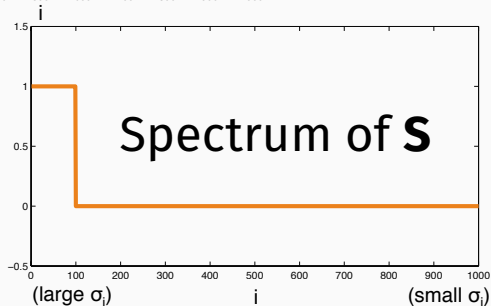
Goal: Apply $\mathbf{A}_\lambda^T$ quickly to any vector.

Goal: Apply $\mathbf{A}_\lambda^T$ quickly to any vector.

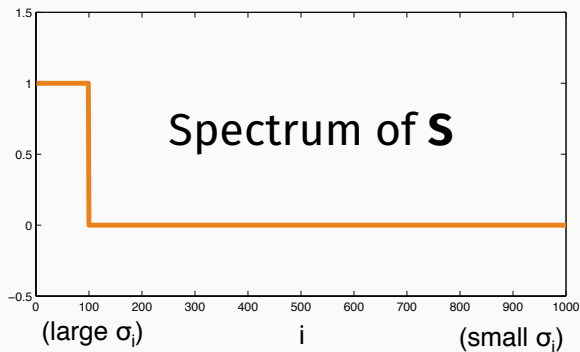




$$\mathbf{A}_{\lambda}^T = \mathbf{S}\mathbf{A}^T$$



$$A_{\lambda}^T \mathbf{b} = \mathbf{S} A^T \mathbf{b}$$



How do we apply $\mathbf{S}$ to $\mathbf{A}^T \mathbf{b}$?

We turn to **Ridge Regression**, a popular alternative to PCR:

We turn to **Ridge Regression**, a popular alternative to PCR:

Goal: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|^2 + \lambda \|\mathbf{x}\|^2$.

Solution: $\mathbf{x}^* = (\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^T \mathbf{b}$.

We turn to **Ridge Regression**, a popular alternative to PCR:

Goal: $\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|^2 + \lambda \|\mathbf{x}\|^2$.

Solution: $\mathbf{x}^* = (\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^T \mathbf{b}$.

Claim:

$$\mathbf{R} = (\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^T \mathbf{A}$$

coarsely approximates $\mathbf{S}$.

Singular values of $\mathbf{S}$:

$$\sigma_i(\mathbf{S}) = \begin{cases} 1 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ 0 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

Singular values of $\mathbf{S}$:

$$\sigma_i(\mathbf{S}) = \begin{cases} 1 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ 0 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

Singular values of $\mathbf{R} = (\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^T \mathbf{A}$:

$$\sigma_i(\mathbf{R}) = \frac{\sigma_i^2(\mathbf{A})}{\sigma_i^2(\mathbf{A}) + \lambda}$$

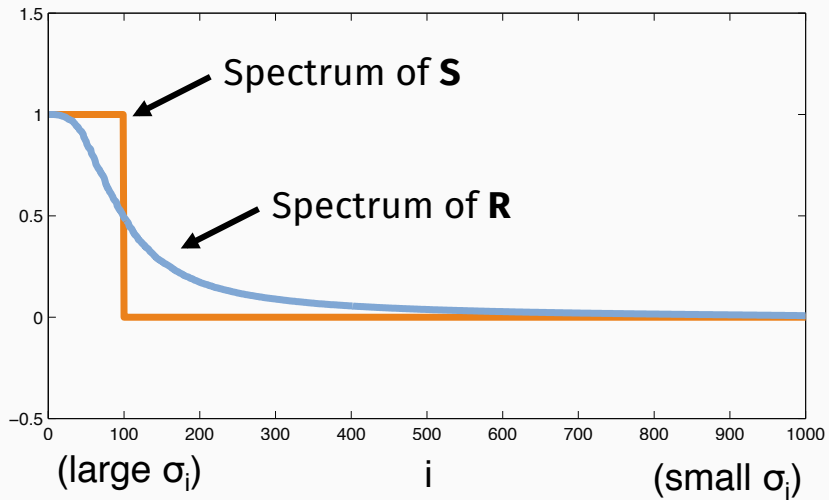
Singular values of $\mathbf{S}$:

$$\sigma_i(\mathbf{S}) = \begin{cases} 1 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ 0 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

Singular values of $\mathbf{R} = (\mathbf{A}^T\mathbf{A} + \lambda\mathbf{I})^{-1}\mathbf{A}^T\mathbf{A}$:

$$\sigma_i(\mathbf{R}) = \frac{\sigma_i^2(\mathbf{A})}{\sigma_i^2(\mathbf{A}) + \lambda} \approx \begin{cases} 1 & \text{if } \sigma_i^2(\mathbf{A}) \gg \lambda, \\ 0 & \text{if } \sigma_i^2(\mathbf{A}) \ll \lambda. \end{cases}$$

A FIRST APPROXIMATION



We just need a way to sharpen this approximation.

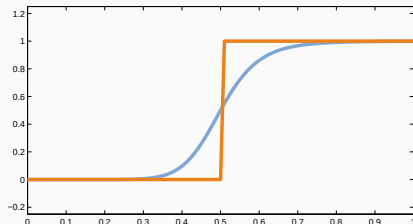
We just need a way to sharpen this approximation.

$$\sigma_i(\mathbf{R}) = \begin{cases} \geq 1/2 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ < 1/2 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

We just need a way to sharpen this approximation.

$$\sigma_i(\mathbf{R}) = \begin{cases} \geq 1/2 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ < 1/2 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

Compose $\mathbf{R}$ with approximate *symmetric* step function:

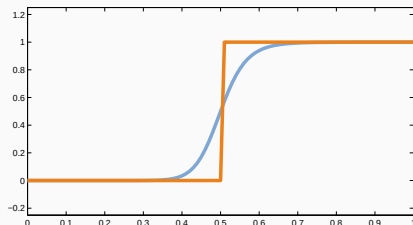


$$f(\mathbf{R}) = c_1 \mathbf{R} + c_2 \mathbf{R}^2 +$$

We just need a way to sharpen this approximation.

$$\sigma_i(\mathbf{R}) = \begin{cases} \geq 1/2 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ < 1/2 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

Compose $\mathbf{R}$ with approximate *symmetric* step function:

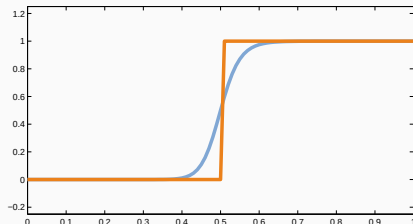


$$f(\mathbf{R}) = c_1\mathbf{R} + c_2\mathbf{R}^2 + c_3\mathbf{R}^3 +$$

We just need a way to sharpen this approximation.

$$\sigma_i(\mathbf{R}) = \begin{cases} \geq 1/2 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ < 1/2 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

Compose $\mathbf{R}$ with approximate *symmetric* step function:

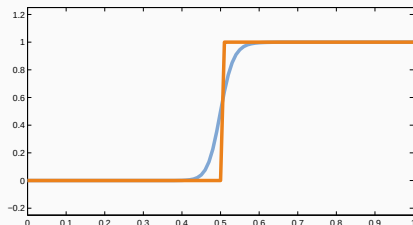


$$f(\mathbf{R}) = c_1\mathbf{R} + c_2\mathbf{R}^2 + c_3\mathbf{R}^3 + c_4\mathbf{R}^4 +$$

We just need a way to sharpen this approximation.

$$\sigma_i(\mathbf{R}) = \begin{cases} \geq 1/2 & \text{if } \sigma_i^2(\mathbf{A}) \geq \lambda, \\ < 1/2 & \text{if } \sigma_i^2(\mathbf{A}) < \lambda. \end{cases}$$

Compose $\mathbf{R}$ with approximate *symmetric* step function:



$$f(\mathbf{R}) = c_1\mathbf{R} + c_2\mathbf{R}^2 + c_3\mathbf{R}^3 + c_4\mathbf{R}^4 + c_5\mathbf{R}^5 + \dots$$

1. Compute $\mathbf{R}\mathbf{A}^T\mathbf{b}$, $\mathbf{R}^2\mathbf{A}^T\mathbf{b}$, $\dots$ $\mathbf{R}^{O(\log 1/\epsilon)}\mathbf{A}^T\mathbf{b}$.

1. Compute $\mathbf{R}\mathbf{A}^T\mathbf{b}$, $\mathbf{R}^2\mathbf{A}^T\mathbf{b}$, $\dots$ $\mathbf{R}^{O(\log 1/\epsilon)}\mathbf{A}^T\mathbf{b}$.
2. Approximate $\mathbf{S}\mathbf{A}^T\mathbf{b} \approx (c_1\mathbf{R} + c_2\mathbf{R}^2 + \dots c_{O(\log 1/\epsilon)}\mathbf{R}^{O(\log 1/\epsilon)})\mathbf{A}^T\mathbf{b}$.

1. Compute $\mathbf{R}\mathbf{A}^T\mathbf{b}$, $\mathbf{R}^2\mathbf{A}^T\mathbf{b}$, $\dots$ $\mathbf{R}^{O(\log 1/\epsilon)}\mathbf{A}^T\mathbf{b}$.
2. Approximate $\mathbf{S}\mathbf{A}^T\mathbf{b} \approx (c_1\mathbf{R} + c_2\mathbf{R}^2 + \dots c_{O(\log 1/\epsilon)}\mathbf{R}^{O(\log 1/\epsilon)})\mathbf{A}^T\mathbf{b}$.
3. Apply $(\mathbf{A}^T\mathbf{A})^{-1}$ to $\mathbf{S}\mathbf{A}^T\mathbf{b}$

1. Compute $\mathbf{R}\mathbf{A}^T\mathbf{b}$, $\mathbf{R}^2\mathbf{A}^T\mathbf{b}$, $\dots$ $\mathbf{R}^{O(\log 1/\epsilon)}\mathbf{A}^T\mathbf{b}$.
 2. Approximate $\mathbf{S}\mathbf{A}^T\mathbf{b} \approx (c_1\mathbf{R} + c_2\mathbf{R}^2 + \dots c_{O(\log 1/\epsilon)}\mathbf{R}^{O(\log 1/\epsilon)})\mathbf{A}^T\mathbf{b}$.
 3. Apply $(\mathbf{A}^T\mathbf{A})^{-1}$ to $\mathbf{S}\mathbf{A}^T\mathbf{b}$
-
1. $O(\log(1/\epsilon))$ calls to a regression algorithm.

1. Compute $\mathbf{R}\mathbf{A}^T\mathbf{b}$, $\mathbf{R}^2\mathbf{A}^T\mathbf{b}$, $\dots$ $\mathbf{R}^{O(\log 1/\epsilon)}\mathbf{A}^T\mathbf{b}$.
 2. Approximate $\mathbf{S}\mathbf{A}^T\mathbf{b} \approx (c_1\mathbf{R} + c_2\mathbf{R}^2 + \dots c_{O(\log 1/\epsilon)}\mathbf{R}^{O(\log 1/\epsilon)})\mathbf{A}^T\mathbf{b}$.
 3. Apply $(\mathbf{A}^T\mathbf{A})^{-1}$ to $\mathbf{S}\mathbf{A}^T\mathbf{b}$
-
1. $O(\log(1/\epsilon))$ calls to a regression algorithm.
 2. Low cost linear combination of vectors.

1. Compute $\mathbf{R}\mathbf{A}^T\mathbf{b}$, $\mathbf{R}^2\mathbf{A}^T\mathbf{b}$, $\dots$ $\mathbf{R}^{O(\log 1/\epsilon)}\mathbf{A}^T\mathbf{b}$.
 2. Approximate $\mathbf{S}\mathbf{A}^T\mathbf{b} \approx (c_1\mathbf{R} + c_2\mathbf{R}^2 + \dots c_{O(\log 1/\epsilon)}\mathbf{R}^{O(\log 1/\epsilon)})\mathbf{A}^T\mathbf{b}$.
 3. Apply $(\mathbf{A}^T\mathbf{A})^{-1}$ to $\mathbf{S}\mathbf{A}^T\mathbf{b}$
-
1. $O(\log(1/\epsilon))$ calls to a regression algorithm.
 2. Low cost linear combination of vectors.
 3. One call to a regression algorithm

I'm leaving out a lot of details...

I'm leaving out a lot of details...

- Analysis of error propagation through approximate operations.
- Recurrence for stable application of symmetric step polynomial.
- More work to make last step stable and fast.

I'm leaving out a lot of details...

- Analysis of error propagation through approximate operations.
- Recurrence for stable application of symmetric step polynomial.
- More work to make last step stable and fast.

But the algorithm itself remains simple!

```

function [x,patb] = fpcr(A, b, lambda, iter)
z = A'*b;
pz = ridgeReg(A,A*z,lambda);

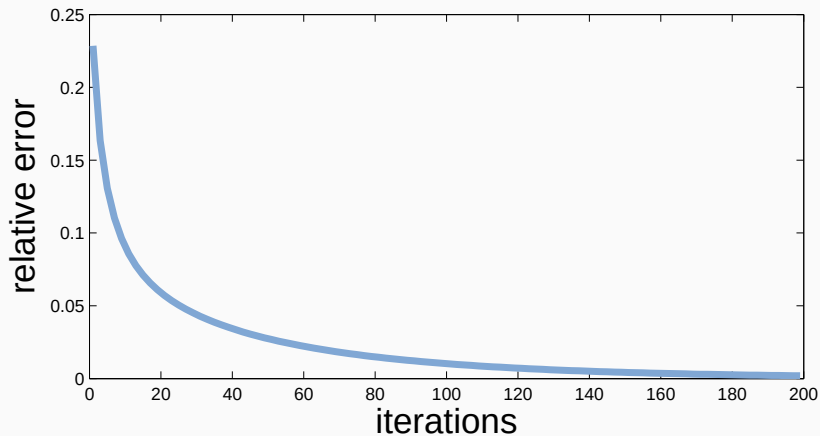
w = pz - z/2;
for i = 1:iter
    w = 4*(2*i+1)/(2*i)*ridgeReg(A, ...
        A*(w - ridgeReg(A,A*w,lambda)), lambda);
    pz = pz + 1/(2*i+1)*w;
end
patb = pz;

x = robustReg(A,pz,lambda);
end

function x = robustReg(A,pz,lambda)
    tol = 1e-5; %default
    function y = afun(z,~)
        y = A'*(A*z) + tol*lambda*z;
    end
    [x,~] = pcg(@afun,pz);
end

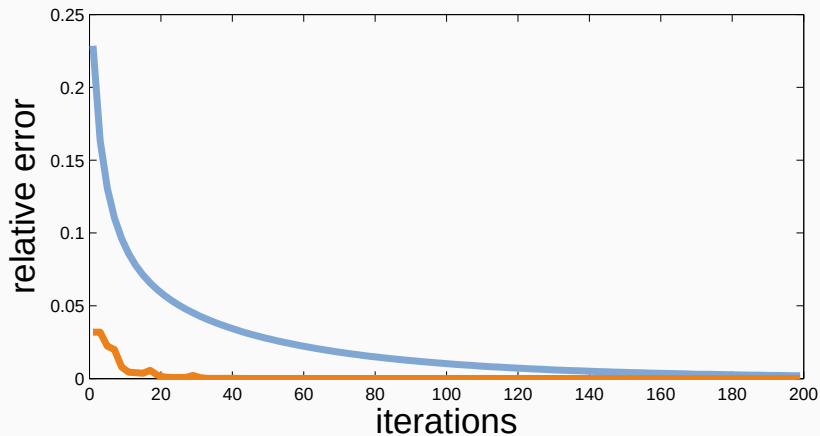
```

Synthetic data (with small spectral gap)



Standard Algorithm

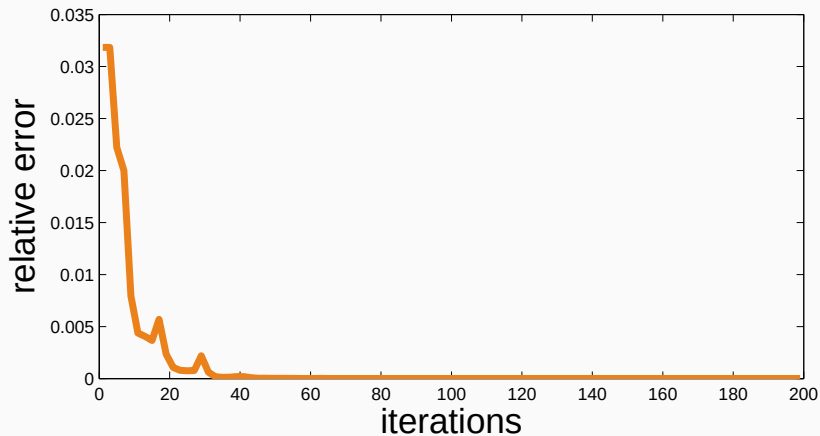
Synthetic data (with small spectral gap)



Standard Algorithm

Krylov Accelerated Algorithm

Synthetic data (with small spectral gap)



Standard Algorithm

Krylov Accelerated Algorithm

Take away: PCR can be computed *without explicit PCA*.

Ridge regression + matrix polynomial = efficient operator
access to $\mathbf{A}$'s top principal components.

Take away: PCR can be computed *without explicit PCA*.

Ridge regression + matrix polynomial = efficient operator
access to $\mathbf{A}$'s top principal components.

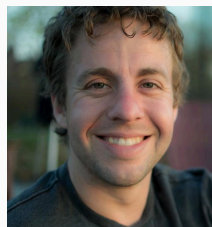
Questions? Joint work with:



Roy Frostig



Cameron Musco



Aaron Sidford